Student handbook

Every lesson in one booklet

This handbook follows the online course lesson by lesson. Use it in class, at home, or when you are offline. The videos, simulator and quizzes are online.

Safety rules

- Ask an adult before using batteries. Use only the battery pack from the kit.
- Never let the battery's red and black wires touch each other.
- Switch off and unplug power before changing any wire.
- Keep fingers, hair and loose clothes away from spinning wheels.
- If anything gets hot, smells or smokes: switch off and tell an adult.

Course map

Module	Lessons
Meet your robot What a robot is, how every robot senses, thinks and acts, and the parts in your kit.	1. Welcome to Robotics 101 (12 min) 2. How do robots think? Sense, Think, Act (14 min) 3. Explore your robot kit (15 min)
How the robot works How IR sensors see the line, how motors steer the robot, and how to build and wire it safely.	4. How IR sensors detect a line (18 min) 5. How motors move a robot (15 min) 6. Build the robot body (16 min) 7. Connect the electronics (18 min)
Program the robot Write Arduino programs that read the sensors, drive the motors and follow the line.	8. Your first Arduino program (15 min) 9. Read the IR sensors (15 min) 10. Control the motors with code (15 min) 11. The line-following algorithm (18 min)
Build, test and improve Run the full program, fix common problems, complete the final challenge and earn your certificate.	12. Make your robot follow the line (17 min) 13. Fix common robot problems (15 min) 14. Final challenge: show your robot (20 min) 15. Final assessment and celebration (15 min)

Welcome to Robotics 101

In this course you will build a robot that follows a black line all by itself, with no remote control. By the end you will know how its sensors see, how its motors move, and how to write the program that connects them.

You will:

- Say what makes a machine a robot
- Spot robots in everyday life in India
- Know the five safety rules for this course
- Know what you will build by the end

What is a robot?

A robot is a machine that can **sense** the world around it, **decide** what to do, and **act** on its own. A ceiling fan is not a robot: it spins whether or not anyone is in the room. An automatic door at a metro station is closer: it senses a person, decides to open, and moves.

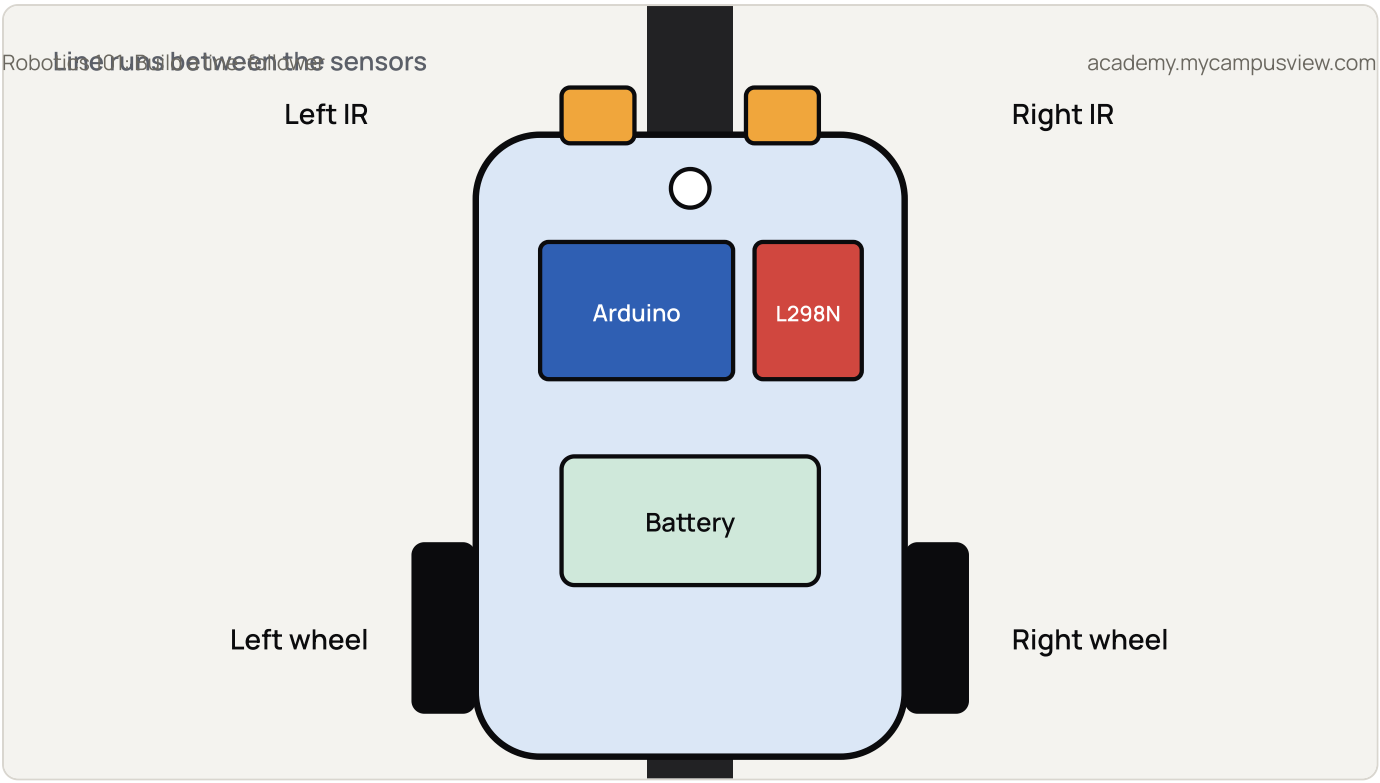
The line-follower you will build does all three. Two small sensors look at the floor, a tiny computer called an Arduino decides which way to steer, and two motors turn the wheels.

Robots around you

Robots sort parcels in warehouses, help doctors in surgery, explore the Moon (the Pragyan rover from Chandrayaan-3 drove on the lunar surface in 2023) and carry goods around factories by following lines painted on the floor, just like yours will.

What you will build

Your robot is a small two-wheeled car. Under its nose are two IR (infrared) sensors that can tell black tape from a white floor. An **Arduino Uno** reads them, and an **L298N motor driver** powers two motors. You will build it, wire it, program it, test it and then improve it.



Your line-follower from above: sensors at the front, wheels at the back.

Five safety rules for every lesson

1. **Unplug the battery** before you touch or change any wire.
2. **An adult checks your wiring** before you connect the battery for the first time.
3. **Never power motors from the Arduino pins or its 5V pin.** Motors get power from the motor driver and battery.
4. **Wheels off the table** when you test motors, so the robot cannot run off the edge.
5. **If anything gets hot, smells or smokes:** unplug the battery at once and tell an adult.

No kit yet?

You can still finish this course. Every build lesson has a virtual version in the simulator, and the final challenge has a virtual pathway your teacher can approve.

What you learned

- A robot senses, decides and acts on its own.
- Robots are all around us, from warehouse carts to Moon rovers.
- You will build a two-wheeled robot that follows a black line.
- Unplug the battery before changing wires, and get an adult to check your wiring.

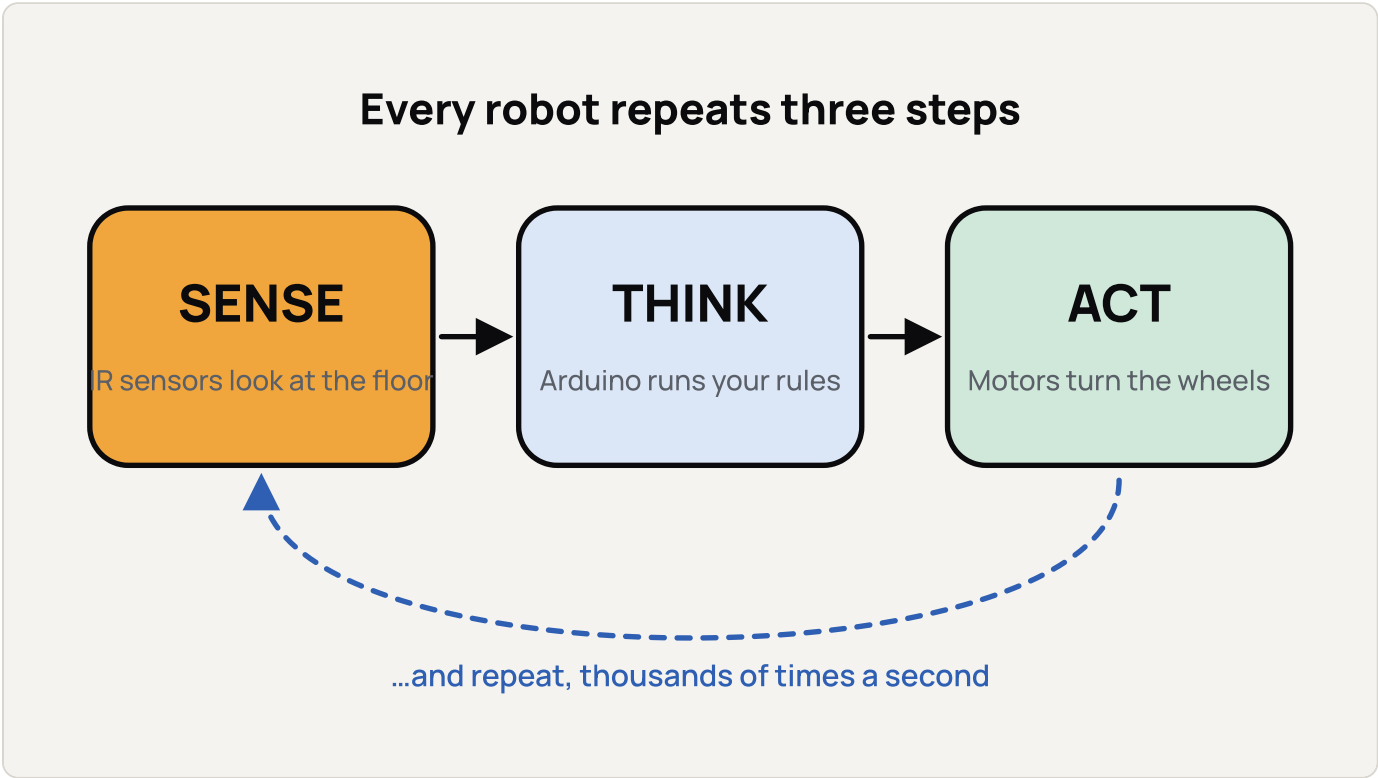
Next: Next, you will look inside a robot's "brain" and learn the Sense, Think, Act loop that every robot follows.

How do robots think? Sense, Think, Act

Every robot, from a toy to a Mars rover, repeats the same three steps over and over: sense, think, act. Once you see this loop, you can understand any robot.

You will:

- Name the three steps every robot repeats
- Match robot parts to Sense, Think and Act
- Explain why the loop must repeat quickly



The loop runs again and again, many times every second.

The three steps

Sense. The robot measures something about the world. Your robot's IR sensors check whether each one is over black tape or white floor.

Think. The robot's computer (the Arduino) compares what it sensed with its rules. For example: "if the left sensor sees black, turn left."

Act. The robot does something. Your robot speeds up or slows down each wheel.

Then it starts again from Sense. Your Arduino can go round this loop thousands of times every second.

You do it too

Crossing a road: you **sense** (look both ways and listen), **think** (is it safe?), and **act** (walk, or wait). Then you keep sensing as you cross.

What you learned

- Every robot repeats Sense, Think, Act.
- Your IR sensors sense, the Arduino and your program think, and the motors act.
- The loop must repeat quickly so the robot can correct itself in time.

Next: Next, open your kit and meet each part: what it is called, what it does, and where it goes.

Explore your robot kit

Before you build, get to know every part. Tap each part of the robot to learn its name and its job, then test yourself.

You will:

- Name the main parts of a line-following robot
- Say what job each part does
- Know which parts sense, which think and which act

Your kit list

- 1 × robot chassis (base plate) with screws
- 2 × DC gear motors (yellow “BO” motors) and 2 × wheels
- 1 × castor wheel (the small ball wheel at the front)
- 1 × Arduino Uno and USB cable
- 1 × L298N motor driver (red or blue board with a heat sink)
- 2 × IR sensor modules (each with a small blue knob)
- 1 × battery holder and batteries (for example 2 × 18650 cells, 7.4 V) with a switch
- Jumper wires (male–female and male–male)

New words

Chassis: the body that everything is fixed to.

Microcontroller: a tiny computer on one chip. The Arduino Uno has one.

Motor driver: a board that lets small signals control big motor currents.

Module: a small ready-made board, like the IR sensor module.

Handle with care

Hold boards by their edges. The pins underneath are sharp, and static from your fingers can harm chips. Keep batteries away from metal tools and loose wires.

What you learned

- The chassis, motors and wheels are the body.
- Two IR sensors sense the line; the Arduino Uno decides.
- The L298N motor driver uses battery power to run the motors.

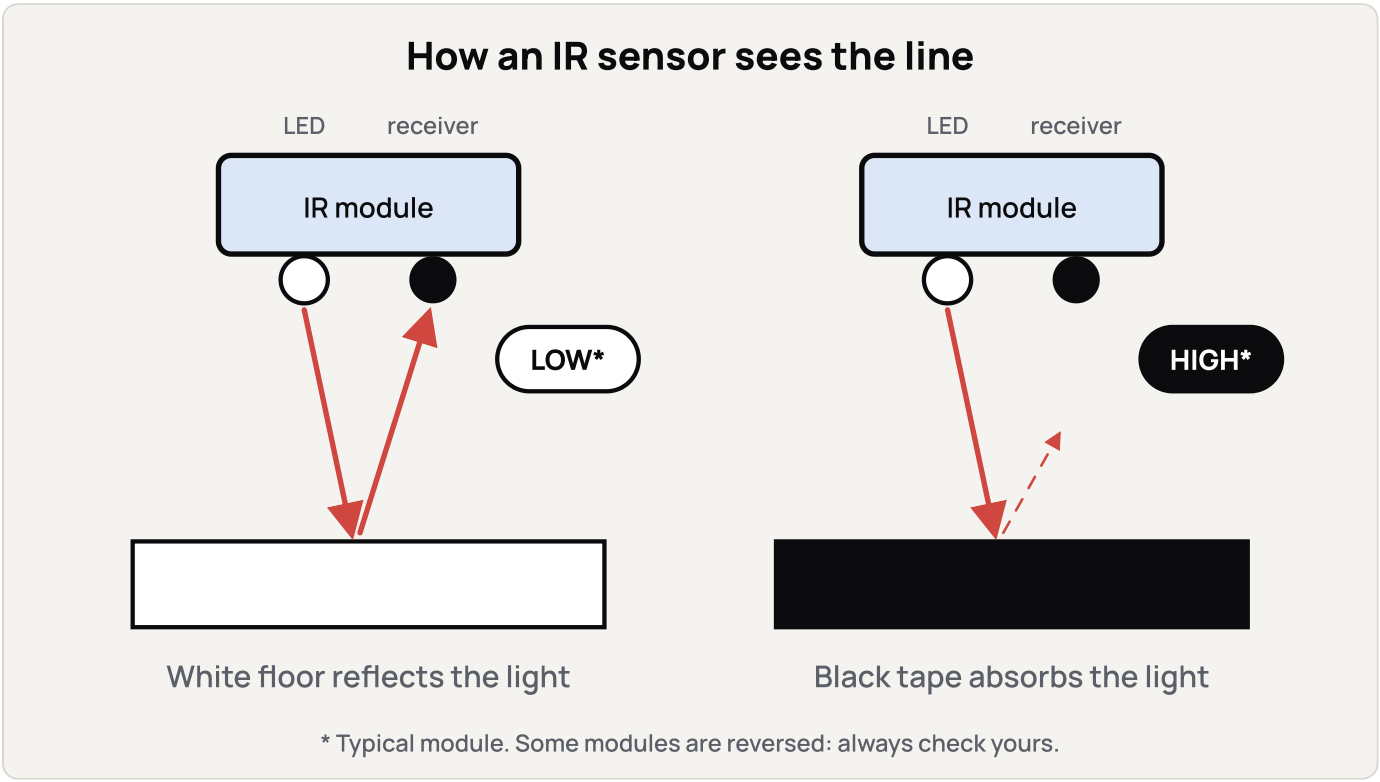
Next: Next, discover how an IR sensor can tell black tape from a white floor using light you cannot see.

How IR sensors detect a line

Your robot cannot see colours like you do. Instead, each IR sensor shines invisible infrared light at the floor and measures how much bounces back. That is enough to find a black line.

You will:

- Explain how an IR sensor uses reflected light
- Predict what a sensor reads over black and over white
- Understand HIGH and LOW, and why some sensors are the other way round
- Know why sensor height and calibration matter



The IR LED sends light down. The receiver measures what comes back.

Light that bounces

Each sensor module has two tiny parts facing the floor: an **IR LED** that sends out infrared light, and an **IR receiver** that measures it.

- A **white** or shiny floor **reflects** most of the light back. The receiver gets a lot.
- **Black** tape **absorbs** most of the light. Very little comes back.

The module compares the amount of light with a set level, called the **threshold**, and sends out a simple answer on its OUT pin: HIGH or LOW.

HIGH and LOW

HIGH means about 5 volts on the pin, which the Arduino reads as 1. **LOW** means about 0 volts, read as 0. A digital pin only ever reports one of these two.

Polarity: not every sensor agrees

Many IR modules send **LOW over white** (lots of reflection) and **HIGH over black**. Some do the opposite. Neither is wrong. That is why in Lesson 9 you will hold each sensor over your tape and **check what it actually reads** before trusting your program.

Calibration

The small blue knob on the module (a **potentiometer**) sets the threshold. Turn it slowly with a small screwdriver while the sensor is over the tape and then over the floor, until the little indicator LED changes exactly when it crosses the edge of the line.

Height matters too. Mount the sensors about **5 to 10 mm** above the floor. Too high and both colours look alike; too low and the sensor scrapes.

Sunlight can confuse it

Sunlight contains lots of infrared. Bright sun on the track can make a sensor read wrongly. Test indoors, away from direct sunlight.

What you learned

- An IR sensor shines invisible light down and measures what bounces back.
- White reflects a lot; black absorbs most of it.
- The module turns this into HIGH or LOW. Some modules are reversed, so always check.
- Calibrate with the blue knob and mount sensors 5 to 10 mm above the floor.

Next: Next, find out how two motors can make a robot go straight, turn and spin.

How motors move a robot

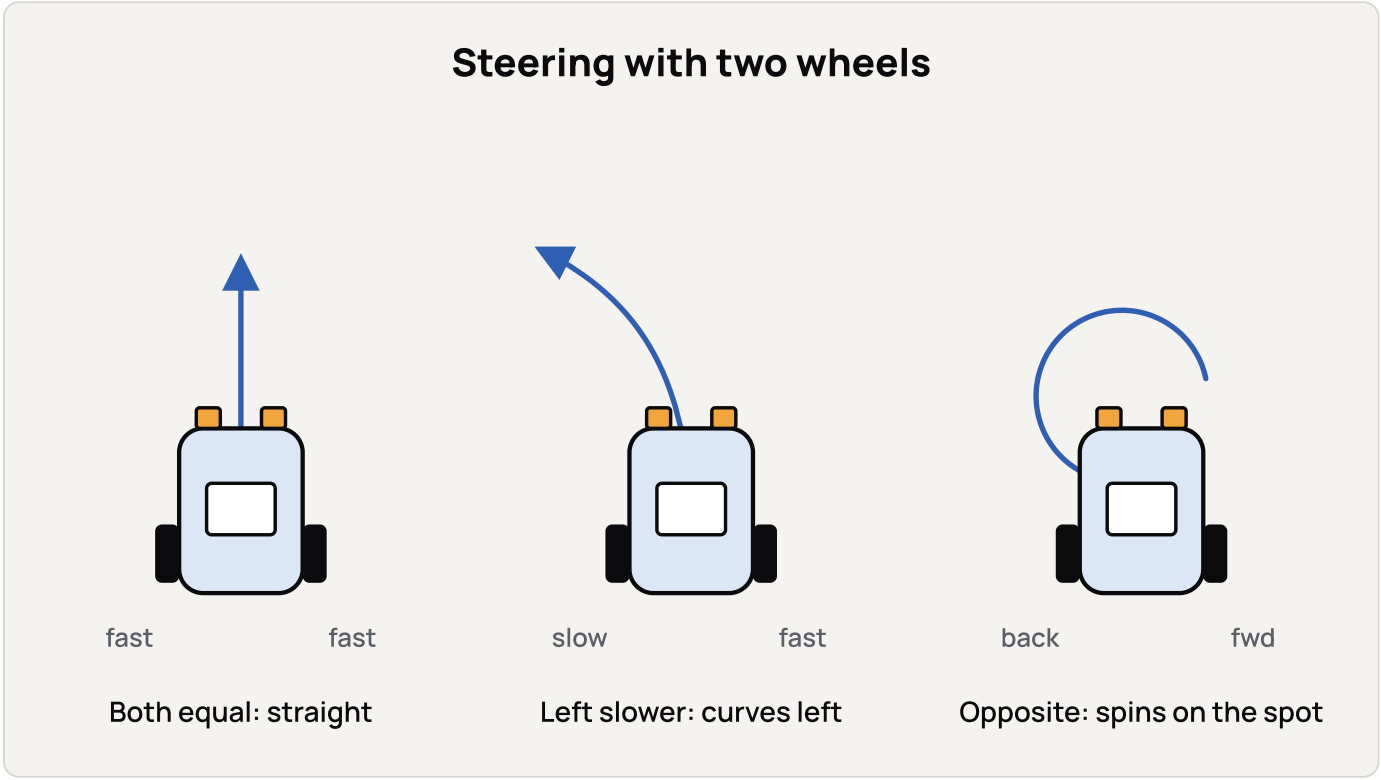
Your robot has no steering wheel. It turns by running its two wheels at different speeds. This is called differential drive, and it is how most small robots move.

You will:

- Explain how a DC motor turns a wheel
- Make a robot go forward, turn and spin using two wheel speeds
- Explain why the Arduino needs a motor driver

DC gear motors

A **DC motor** spins when current flows through it. Swap the two wires and it spins the other way. The yellow motors in your kit have a **gearbox** that trades speed for strength, so the wheels turn slower but can push the robot.



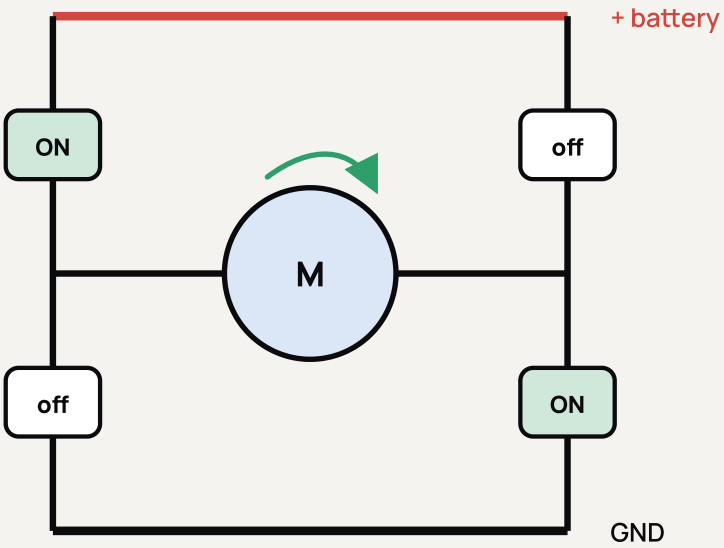
Equal speeds go straight. A slower wheel is on the inside of the turn.

Why a motor driver?

An Arduino pin can safely give about **20 milliamps**. A small motor can need **several hundred**. Connect a motor straight to a pin and you can damage the Arduino.

The **L298N** solves this. The Arduino sends it small signals (which direction, how fast), and it switches the much bigger battery current to the motors. Inside it is a circuit called an **H-bridge** that can reverse the current, so the motor can run backwards too.

Inside the driver: an H-bridge



One diagonal pair on: current flows one way, motor turns forward

An H-bridge: closing opposite switches sends current one way or the other.

What you learned

- A DC motor spins one way or the other depending on which way current flows.
- Equal wheel speeds go straight; a slower wheel turns the robot towards that side.
- Opposite directions spin the robot on the spot.
- Arduino pins are too weak for motors, so the L298N driver supplies the current.

Next: Next, put the robot's body together, step by step.

Build the robot body

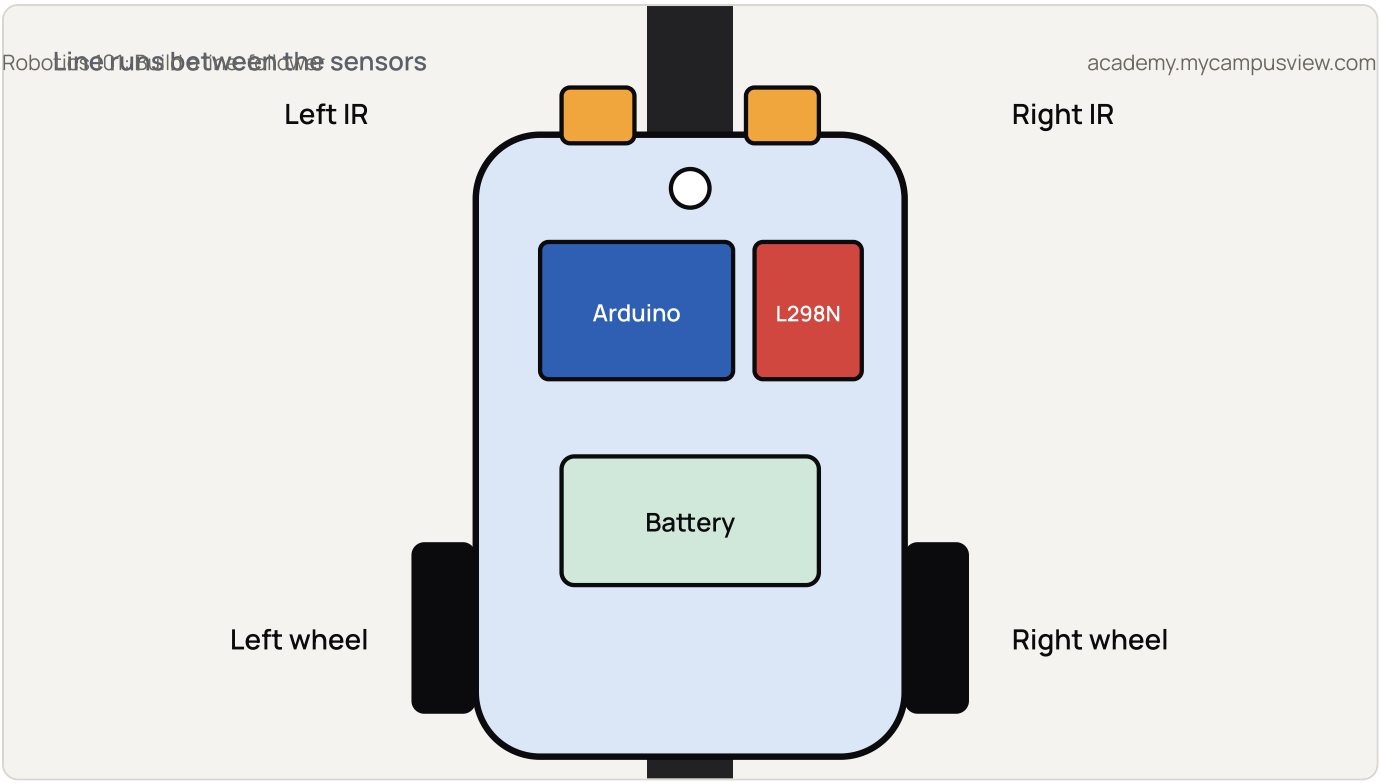
Time to build. Follow each step, tick it off, and check your work before moving on. Take your time: a solid body makes everything later easier.

You will:

- Mount the motors, wheels and castor on the chassis
- Fix the Arduino, motor driver, sensors and battery holder
- Check the build before any wiring

Assembly guide

1. **Lay out every part.** Check everything on the kit list is there before you start. Keep screws in a small bowl.
2. **Fix the two motors.** Screw each motor to the underside of the chassis at the back, shafts pointing outwards. The wires should face inwards. *Tip: Tighten until firm, not as hard as you can. Plastic brackets crack.*
3. **Push on the wheels.** Line up the flat side of the shaft with the wheel's hole and press straight on.
4. **Mount the castor.** Fix the castor wheel under the front so the chassis sits level.
5. **Fix the sensors.** Mount both IR modules at the front edge, facing down, about 5 to 10 mm above the floor and about 3 cm apart, one either side of where the line will be. *Tip: Your black tape should fit between the two sensors with a little gap on each side.*
6. **Mount the Arduino and driver.** Fix the Arduino Uno on spacers on top of the chassis, with the USB port reachable. Fix the L298N next to it.
7. **Fit the battery holder.** Fix the holder where it balances the robot, usually over the back wheels. Do not put batteries in yet.



Top view: sensors at the front, Arduino and driver in the middle, battery at the back.

Build check

- [ ] Both motors are firmly fixed and do not wobble
- [ ] Both wheels spin freely by hand without rubbing
- [ ] The robot sits level on its wheels and castor
- [ ] Both IR sensors face down, 5 to 10 mm above the floor
- [ ] The Arduino's USB port can be reached
- [ ] No batteries are in the holder yet

No kit? Build it virtually

Open the simulator in Lesson 11 and change the sensor spacing and line width to see why sensor placement matters.

What you learned

- Motors at the back, castor at the front, sensors at the front edge facing down.
- Sensors sit 5 to 10 mm above the floor, either side of the line.
- Batteries stay out until the wiring has been checked.

Next: Next, connect the electronics: sensors and motor driver to the Arduino, with a common ground.

Connect the electronics

Now connect the parts so they can talk. The sensors send signals to the Arduino, the Arduino sends signals to the motor driver, and the battery powers the motors through the driver.

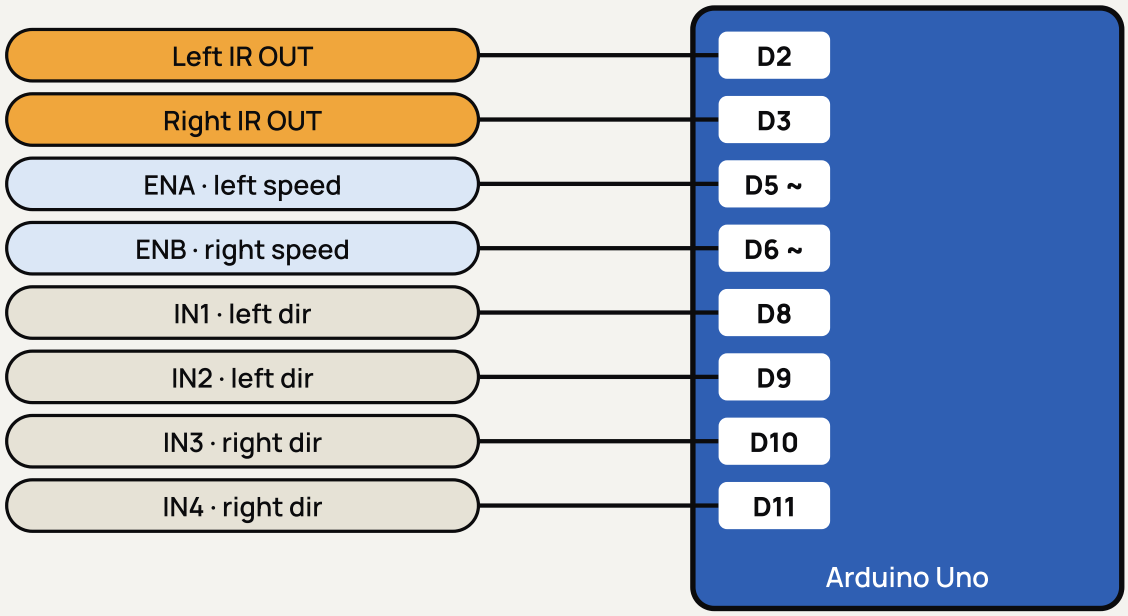
You will:

- Connect both IR sensors and the L298N to the correct Arduino pins
- Explain why every part must share a common ground
- Pass a wiring check with an adult before connecting power

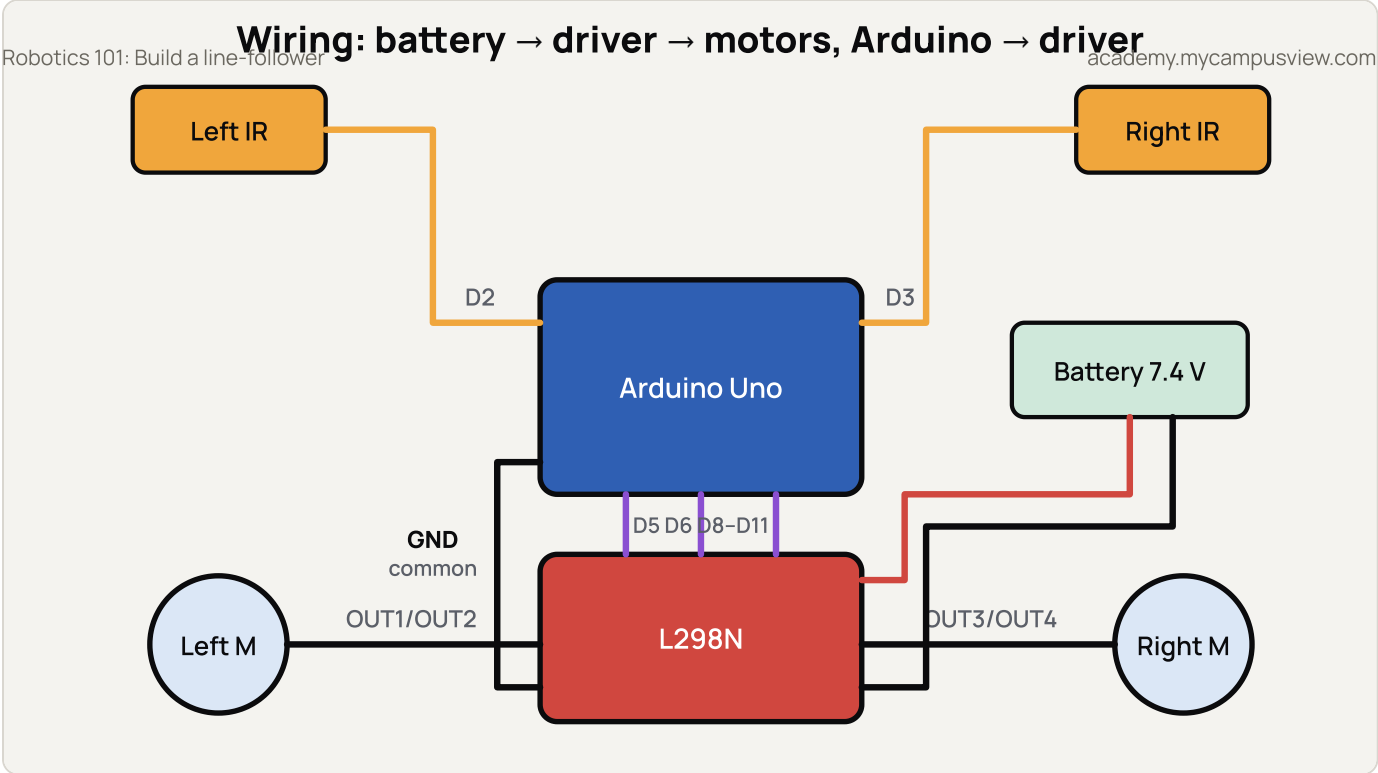
Before you start

Batteries out. USB cable unplugged. You will connect power only at the very end, after an adult has checked every wire.

Reference pin map



The reference pin map used in every sketch in this course.



Full wiring. Red: power. Black: ground. Other colours: signals.

Power and ground

- Battery **+** goes to the L298N **12V** (or VS) terminal through the switch. Battery **−** goes to the L298N **GND** terminal.
- Run a wire from the L298N **GND** to an Arduino **GND** pin. This is the **common ground**.
- Each IR module: **VCC to 5V**, **GND to GND**, **OUT to its Arduino pin**.
- Left motor wires go to L298N **OUT1/OUT2** (motor A); right motor to **OUT3/OUT4** (motor B).
- Remove the jumper caps on **ENA** and **ENB** so the Arduino can control speed through them.

Common ground

A voltage is always measured between two points. If the Arduino and the driver do not share the same ground wire, a 5V signal from the Arduino means nothing to the driver, and the motors behave randomly or not at all. **Connect all the grounds together.**

Powering the Arduino

Many L298N boards have a 5V regulator and a 5V terminal. Whether you can use it to power the Arduino depends on your board and battery. For this course, power the Arduino from **USB while programming and testing**, and follow your kit's instructions for running on battery. Never connect a battery above 12 V.

Adult wiring check (before power)

- [ ] Left IR OUT to D2, right IR OUT to D3
- [ ] Both IR VCC to 5V and GND to GND
- [ ] ENA to D5, ENB to D6 (jumper caps removed)

- [ ] IN1 D8, IN2 D9, IN3 D10, IN4 D11
- [ ] L298N GND connected to Arduino GND (common ground)
- [ ] Battery + to L298N 12V/VS through the switch, battery − to L298N GND
- [ ] No motor wire goes to any Arduino pin
- [ ] No bare wire ends touching each other

What you learned

- Sensors: VCC 5V, GND, OUT to D2 (left) and D3 (right).
- Driver: ENA D5, ENB D6, IN1–IN4 on D8–D11. Motors on OUT1–OUT4.
- All grounds connect together: battery, driver and Arduino.
- An adult checks the wiring before the battery goes in.

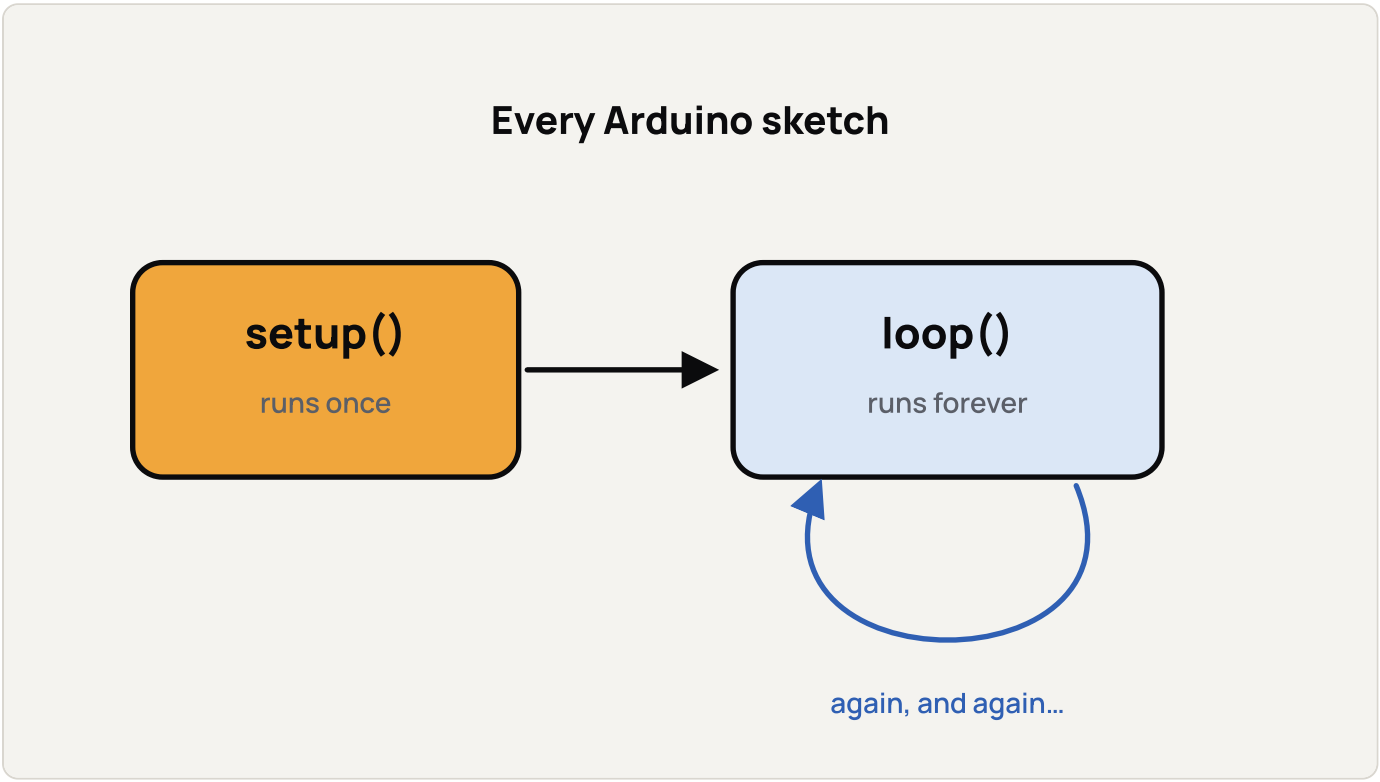
Next: Next, write your very first Arduino program.

Your first Arduino program

An Arduino program is called a sketch. Every sketch has two parts: setup(), which runs once, and loop(), which runs forever. Here is how to write, upload and run your first one.

You will:

- Explain what setup() and loop() do
- Use a variable to store a number
- Use pinMode, digitalWrite, delay and Serial.println
- Upload a sketch and read the Serial Monitor



setup() runs once. loop() runs again and again.

```
// Robotics 101, Lesson 8: your first Arduino program
// Blinks the built-in LED and says hello on the Serial Monitor.
// Board: Arduino Uno. Nothing else needs to be connected.

int blinkCount = 0;          // a variable: a named box that holds a number

void setup() {
  // setup() runs once, when the board powers on or resets.
  pinMode(LED_BUILTIN, OUTPUT);   // pin 13 has a small LED on the board
  Serial.begin(9600);             // open the Serial Monitor at 9600 baud
  Serial.println("Hello! My robot brain is awake.");
}

void loop() {
  // loop() runs again and again, forever.
  digitalWrite(LED_BUILTIN, HIGH);  // LED on
  delay(500);                       // wait half a second (500 milliseconds)
  digitalWrite(LED_BUILTIN, LOW);   // LED off
  delay(500);

  blinkCount = blinkCount + 1;
  Serial.print("Blinks so far: ");
  Serial.println(blinkCount);
}
```

Upload it

1. Install the free **Arduino IDE** from arduino.cc on a computer.
2. Plug the Arduino Uno in with its USB cable.
3. Choose **Tools → Board → Arduino Uno** and **Tools → Port** (the one that appears when you plug in).
4. Paste the sketch and click **Upload** (the right arrow).
5. Open **Tools → Serial Monitor** and set it to **9600 baud**.

The small LED marked **L** blinks, and the Serial Monitor counts the blinks.

Common mistakes

- Forgetting the **semicolon** ; at the end of a line.
- Writing **Setup** with a capital S. Arduino code is case-sensitive.
- The Serial Monitor shows rubbish: set it to **9600 baud** to match Serial.begin(9600).

What you learned

- setup() runs once; loop() repeats forever.
- A variable is a named box that holds a value.
- digitalWrite sets a pin HIGH or LOW; delay waits in milliseconds.
- Serial.println sends text to the Serial Monitor at 9600 baud.

Next: Next, read your IR sensors and see their values on the Serial Monitor.

Read the IR sensors

Now connect your program to the real world. You will read each IR sensor with digitalRead, print what it sees, and find out which value your sensors give over black.

You will:

- Use pinMode(pin, INPUT) and digitalRead
- Print sensor values on the Serial Monitor
- Calibrate: find your sensor's value over black and set ON_LINE

```
// Robotics 101, Lesson 9: read both IR sensors
// Prints what each sensor sees on the Serial Monitor (9600 baud).
// Wiring: left IR OUT -> D2, right IR OUT -> D3, both VCC -> 5V, both GND -> GND.

const int LEFT_IR = 2;
const int RIGHT_IR = 3;

// Many IR modules output HIGH over black and LOW over white, but some are the
// other way round. Hold each sensor over your black tape: if the Serial Monitor
// says WHITE, change this to LOW.
const int ON_LINE = HIGH;

void setup() {
  pinMode(LEFT_IR, INPUT);
  pinMode(RIGHT_IR, INPUT);
  Serial.begin(9600);
  Serial.println("IR sensor test. Move the robot over the line.");
}

void loop() {
  int left = digitalRead(LEFT_IR);
  int right = digitalRead(RIGHT_IR);

  Serial.print("Left: ");
  Serial.print(left == ON_LINE ? "BLACK" : "WHITE");
  Serial.print("   Right: ");
  Serial.println(right == ON_LINE ? "BLACK" : "WHITE");

  delay(200);  // slow enough to read
}
```

Calibrate your sensors

1. **Both on white.** Place the robot on plain white floor. Both should read WHITE, and each module's small LED should be in the same state.
2. **Left on black.** Slide the robot so only the left sensor is over the tape. The Serial Monitor should say Left: BLACK.

3. **If it says WHITE over black.** Your modules use the opposite polarity. Change const int ON_LINE = HIGH; to LOW, upload again and re-check. *Tip: This is normal. Not every module agrees.*
4. **Adjust the knob if needed.** If a sensor never changes, turn its blue knob slowly until its LED switches exactly at the edge of the tape.
5. **Right on black.** Repeat with the right sensor. Note your ON_LINE value: you will use it in Lesson 12.

What you learned

- Sensor pins are INPUTs; read them with digitalRead.
- Serial.print shows what the robot senses.
- Calibrate: set ON_LINE to the value your sensors give over black.

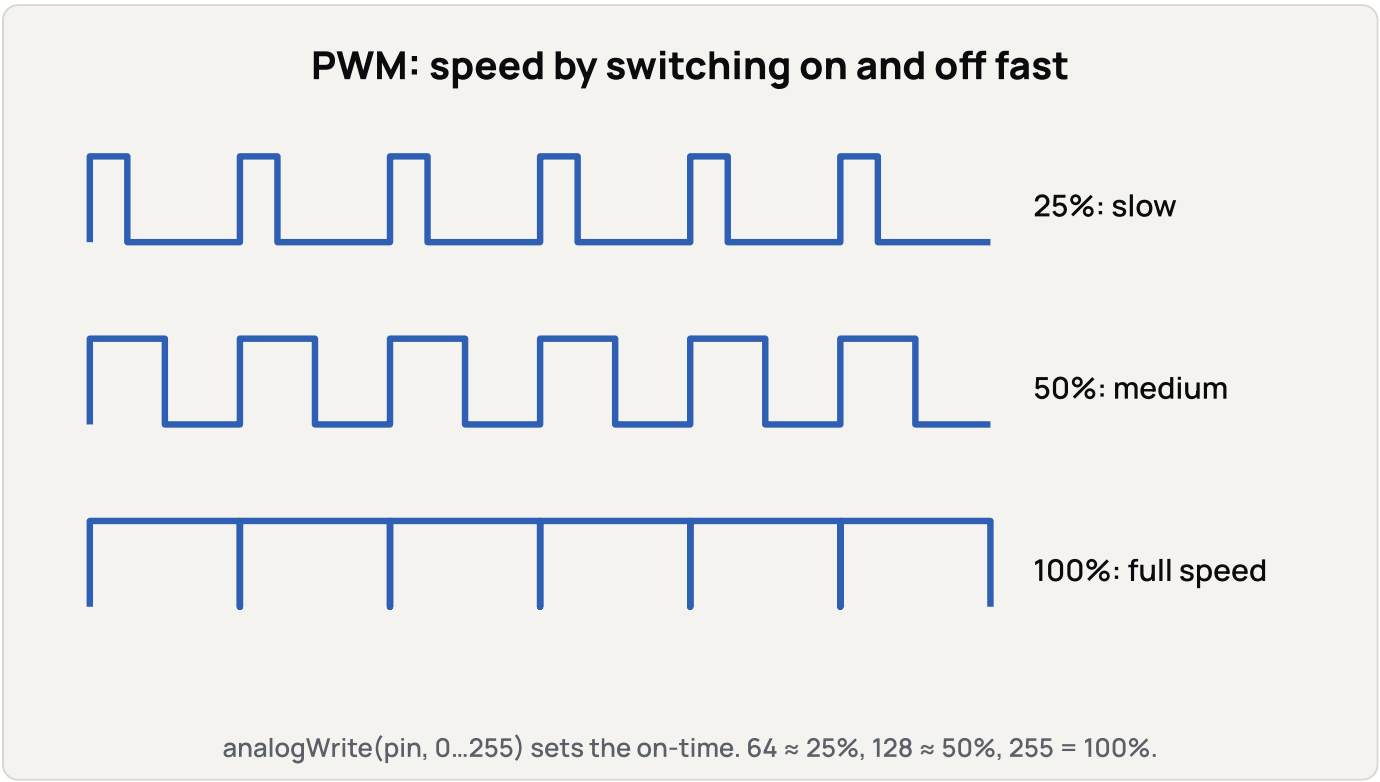
Next: Next, control the motors from code and learn how PWM sets their speed.

Control the motors with code

Two kinds of signal drive each motor. Direction pins (IN1 to IN4) choose which way it turns. A speed pin (ENA or ENB) uses PWM to choose how fast.

You will:

- Set motor direction with IN1–IN4
- Set motor speed with analogWrite and PWM
- Make the robot go forward, turn left, turn right and stop



PWM: the pin switches on and off very fast. More “on” time means more speed.

Direction and speed

IN1	IN2	Left motor
HIGH	LOW	Forward
LOW	HIGH	Backward
LOW	LOW	Stop

IN3 and IN4 do the same for the right motor.

Speed comes from **analogWrite(ENA, value)**, where value is 0 (stopped) to 255 (full speed). This only works on pins marked **~** on the Uno, such as D5 and D6.

```
// Robotics 101, Lesson 10: test both motors
// Runs a short, safe pattern: forward, stop, left, stop, right, stop, then waits.
// Put the robot on a stand or hold it with the wheels off the table first.
// Wiring (L298N): ENA D5, IN1 D8, IN2 D9 (left motor); ENB D6, IN3 D10, IN4 D11 (right motor).

const int ENA = 5;   // left motor speed (PWM)
const int IN1 = 8;
const int IN2 = 9;
const int ENB = 6;   // right motor speed (PWM)
const int IN3 = 10;
const int IN4 = 11;

const int SPEED = 150;  // 0 (stopped) to 255 (full speed)

void setup() {
  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(ENB, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
  stopMotors();
  Serial.begin(9600);
  Serial.println("Motor test starts in 3 seconds. Wheels off the table!");
  delay(3000);
}

void loop() {
  Serial.println("Forward");
  setMotors(SPEED, SPEED);
  delay(1000);
  stopMotors();
  delay(800);

  Serial.println("Turn left (right wheel only)");
  setMotors(0, SPEED);
  delay(800);
  stopMotors();
  delay(800);

  Serial.println("Turn right (left wheel only)");
  setMotors(SPEED, 0);
  delay(800);
  stopMotors();

  Serial.println("Done. Press RESET to run again.");
  while (true) {
    // stay stopped
  }
}

// Positive speed: forward. Negative: backward. 0: stop.
void setMotors(int leftSpeed, int rightSpeed) {
  driveOne(ENA, IN1, IN2, leftSpeed);
```

```
  driveOne(ENB, IN3, IN4, rightSpeed);
}

void driveOne(int enPin, int inA, int inB, int speed) {
  if (speed > 0) {
    digitalWrite(inA, HIGH);
    digitalWrite(inB, LOW);
  } else if (speed < 0) {
    digitalWrite(inA, LOW);
    digitalWrite(inB, HIGH);
  } else {
    digitalWrite(inA, LOW);
    digitalWrite(inB, LOW);
  }
  analogWrite(enPin, constrain(abs(speed), 0, 255));
}

void stopMotors() {
  setMotors(0, 0);
}
```


A wheel spins backwards?

Swap the two wires of that motor at the L298N terminals, or swap HIGH and LOW for its IN pins in setMotors. Both fix it.

What you learned

- IN pins choose direction; EN pins choose speed.
- analogWrite(pin, 0–255) sets speed using PWM on ~ pins.
- Turning means making one wheel slower than the other.

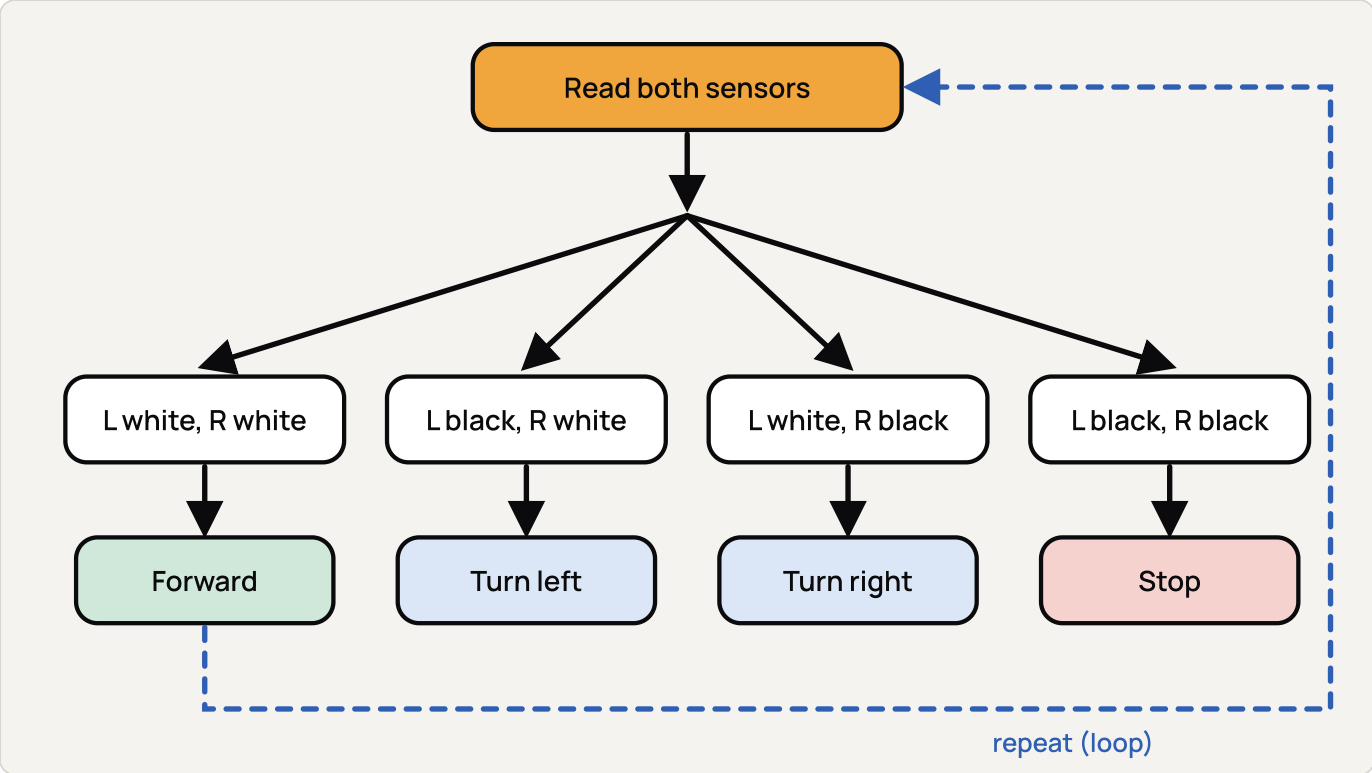
Next: Next, combine sensors and motors into the line-following algorithm.

The line-following algorithm

An algorithm is a set of rules for solving a problem. The line-following algorithm needs just four rules, one for each combination of the two sensors. Learn them, then test them in the simulator.

You will:

- Write the four line-following rules as IF / ELSE
- Complete the truth table for two sensors
- Test the algorithm in the simulator and fix a broken version



The decision the robot makes on every pass of loop().

From rules to code

```
if (!leftOnLine && !rightOnLine) {
  // forward
} else if (leftOnLine && !rightOnLine) {
  // turn left
} else if (!leftOnLine && rightOnLine) {
  // turn right
} else {
  // both on black: stop
}
```

&& means AND. **!** means NOT. So `!leftOnLine && rightOnLine` reads: “the left does not see the line, and the right does.”

What you learned

- Two sensors give four combinations, so four rules.
- Neither on line: forward. Left on line: turn left. Right on line: turn right. Both: stop.
- In code: if / else if / else, with && (AND) and ! (NOT).
- Swapping left and right makes the robot steer away from the line.

Next: Next, put the full program on your robot and make it follow a real track.

Make your robot follow the line

Everything comes together. Upload the full line-follower program, make a test track, and run your robot. Then tune the speed until it is smooth and reliable.

You will:

- Make a safe test track
- Upload and run the full line-follower program
- Tune BASE_SPEED and TURN_SPEED for your track

Make a test track

1. **Choose a surface.** A large white chart paper, flex sheet or a light, smooth floor. Avoid shiny tiles in bright sunlight.
2. **Lay the line.** Use black electrical (insulation) tape, about 18 to 25 mm wide. Start with a large oval with gentle curves. *Tip: Press the tape down flat. Bubbles and creases confuse the sensors.*
3. **Keep it clear.** Leave at least 15 cm of white around the line, and keep the track away from table edges.

```
// Robotics 101, Lesson 12: line-follower
// Two IR sensors either side of a black line; an L298N drives two DC motors.
// Pins: left IR D2, right IR D3, ENA D5, ENB D6, IN1 D8, IN2 D9, IN3 D10, IN4 D11.

const int LEFT_IR = 2;
const int RIGHT_IR = 3;
const int ENA = 5;
const int IN1 = 8;
const int IN2 = 9;
const int ENB = 6;
const int IN3 = 10;
const int IN4 = 11;

// Calibrate in Lesson 9: the value a sensor reads over the black line.
const int ON_LINE = HIGH;

// Tune these in Lesson 12. Lower is easier to control; higher is faster.
const int BASE_SPEED = 150;  // straight driving (0 to 255)
const int TURN_SPEED = 40;   // inner wheel while turning; 0 or negative (e.g. -80) for sharp
corners

// What to do when both sensors see black: STOP at a finish line,
// or keep going to cross a junction or a sharp corner.
const bool STOP_WHEN_BOTH = true;

void setup() {
  pinMode(LEFT_IR, INPUT);
  pinMode(RIGHT_IR, INPUT);
  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(ENB, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
  setMotors(0, 0);
  Serial.begin(9600);
  delay(2000);  // two seconds to put the robot on the track
}

void loop() {
  bool leftOnLine = digitalRead(LEFT_IR) == ON_LINE;
  bool rightOnLine = digitalRead(RIGHT_IR) == ON_LINE;

  if (!leftOnLine && !rightOnLine) {
    // Line between the sensors: go straight.
    setMotors(BASE_SPEED, BASE_SPEED);
  } else if (leftOnLine && !rightOnLine) {
    // Line under the left sensor: turn left by slowing the left wheel.
    setMotors(TURN_SPEED, BASE_SPEED);
  } else if (!leftOnLine && rightOnLine) {
    // Line under the right sensor: turn right by slowing the right wheel.
    setMotors(BASE_SPEED, TURN_SPEED);
  } else {
    // Both on black: a cross line or the finish.
```

```
    if (STOP_WHEN_BOTH) {
      setMotors(0, 0);
    } else {
      setMotors(BASE_SPEED, BASE_SPEED);
    }
  }
}

// Positive speed: forward. Negative: backward. 0: stop.
void setMotors(int leftSpeed, int rightSpeed) {
  driveOne(ENA, IN1, IN2, leftSpeed);
  driveOne(ENB, IN3, IN4, rightSpeed);
}

void driveOne(int enPin, int inA, int inB, int speed) {
  if (speed > 0) {
    digitalWrite(inA, HIGH);
    digitalWrite(inB, LOW);
  } else if (speed < 0) {
    digitalWrite(inA, LOW);
    digitalWrite(inB, HIGH);
  } else {
    digitalWrite(inA, LOW);
    digitalWrite(inB, LOW);
  }
  analogWrite(enPin, constrain(abs(speed), 0, 255));
}
```

Before the first run

- [ ] ON_LINE matches my calibration from Lesson 9
- [ ] The motor test (Lesson 10) makes both wheels go forward
- [ ] Batteries are charged
- [ ] The robot starts with the line between its two sensors
- [ ] An adult is nearby for the first run

Tune it

Start slow. Set **BASE_SPEED** to about 120 and see if the robot completes a lap. Then raise it by 10 or 20 at a time.

- **Wobbles a lot on straights:** turns are too sharp. Raise TURN_SPEED a little.
- **Runs off on curves:** it is too fast for its turns. Lower BASE_SPEED, or lower TURN_SPEED so turns are sharper.
- **Cannot get round a 90° corner:** set TURN_SPEED to 0, or a negative number such as -80, so the inside wheel stops or spins backwards. This is a **pivot turn**.
- **Stops at a cross line you want it to drive over:** set STOP_WHEN_BOTH to false.

Try the same changes in the simulator first: it uses the same rules.

What you learned

- Use 18–25 mm black tape on a smooth white surface.
- Start slow, then raise the speed in small steps.
- Tuning is a balance: faster runs need sharper, quicker turns.

Next: Next, solve the most common problems like a real robotics engineer.

Fix common robot problems

Every engineer's robot misbehaves at first. Good engineers find the cause step by step. Solve these missions to learn the most common faults and how to fix them.

You will:

- Diagnose a problem from its symptom
- Check one thing at a time
- Fix the six most common line-follower faults

The engineer's method

1. **Describe** exactly what happens.
2. **Guess** one likely cause.
3. **Test** just that one thing.
4. **Fix** it, then test again.

Always unplug the battery before changing any wires.

What you learned

- Describe, guess, test, fix: one change at a time.
- No power: check the battery and switch. Random motors: check common ground.
- Wrong direction: swap that motor's wires. Ignores the line: calibrate.
- Loses the line on curves: slow down or turn more sharply.

Next: Next: the final challenge. Show what your robot can do.

Final challenge: show your robot

This is your moment. Show that your robot can follow a line all by itself. Work through the ten steps, then hand in your evidence for your teacher to review.

You will:

- Complete a full lap of a track with two or more curves
- Hand in evidence: photo, code and video, a teacher-witnessed run, or simulator results
- Explain one problem you solved

Ten steps to the finish

- [ ] Assemble the robot
- [ ] Connect the components
- [ ] Test the sensors
- [ ] Test the motors
- [ ] Upload the line-following program
- [ ] Calibrate the sensors
- [ ] Run the robot on the track
- [ ] Fix any problems
- [ ] Improve speed or reliability
- [ ] Record and submit your result

What you learned

- Finish a lap with at least two curves, untouched.
- Submit a photo, code and video, a teacher-witnessed run, or simulator results.
- Your teacher reviews it; you can improve and resubmit.

Next: Last step: the final assessment, your badge and your certificate.

Final assessment and celebration

Ten questions on everything you have learned. You need 80% to pass, and you can try again as many times as you like. Your best score counts.

You will:

- Show what you know about sensors, motors, wiring and code
- Earn the Line-Follower Champion badge and your certificate

Where next?

After this course, try **AI for Young Builders** or an obstacle-avoiding robot with an ultrasonic sensor. Everything you learned here, Sense, Think, Act, carries over.

What you learned

- You built, wired, programmed, tested and improved a real robot.
- Sense, think, act: the same loop runs inside warehouse robots, drones and self-driving cars.

Next: You have finished Robotics 101. Download your certificate from your learning page.